

```
$ apexlang --demo webinar
```

Your First APEXlang App From Nothing to Running in a Single Session

One terminal. One prompt. One live app on Oracle Cloud.

Marco Pereira · markuspg.com · 2026

WITH

ORACLE



ORAPUB
POWERED BY VISCOSITY

Everything on the table.

ON THIS MAC

- [] Terminal + a coding agent – Antigravity today; Claude Code, Cursor, Aider all work
- [] Homebrew – the only package manager we touch
- [] OCI CLI – provisioning, wallets, everything cloud-side – no console
- [] SQLcl 26.1 – Oracle's database CLI; the APEXlang compiler lives inside it
- [] Java 21 – SQLcl's bundled JRE is too old for the APEXlang compiler
- [] Oracle's agent skills – grammar, templates and compiler queries the agent reads instead of guessing

Every box gets checked in the next hour, in this order, from this terminal.

IN ORACLE CLOUD

- [] An OCI account – with one Always Free database slot open
- [] Autonomous Database 19c – provisioned live from the terminal, about a minute
- [] APEX 26.1 – already inside the database – nothing to install
- [] A workspace + schema – created from SQL, no console

THE APP ITSELF

- [] An APEXlang project – .apx files: app, pages, reports, columns, charts
- [] A sales table + 20 rows – shipped as a supporting-object install script
- [] One plain-English prompt – the agent does the rest

Zero tools. Sixty minutes. One live app.

- ▶ Nothing installed at start – assume a fresh Mac
- ▶ Terminal only – no clicking around a cloud console
- ▶ Under one hour, wall-clock
- ▶ End state: a real Oracle APEX application, deployed on Oracle Cloud, that you can share the URL of

Everything you're about to see happens in this window. The only browser tab we'll open is the one that shows the finished app.

Give your LLM a terminal.

```
# any coding agent with shell access works. today: Antigravity.  
$ bash scripts/01-check-agent.sh
```

The moment the LLM can run `real` commands and read `real` output, you stop copy-pasting. You become the `director`. The agent becomes the operator. Claude Code, Cursor Agent, Aider — same idea, pick your poison.

Install the OCI CLI.

```
$ bash scripts/02-install-oci-cli.sh  
# wraps `brew install oci-cli` – skips if already installed
```

This is how the terminal (and therefore the agent) talks to Oracle Cloud Infrastructure. Provisioning databases, downloading wallets, listing compartments – all of it, no console.

Install SQLcl.

```
$ bash scripts/03-install-sqlcl.sh  
# wraps `brew install --cask sqlcl` + PATH symlink – skips if present
```

Oracle's modern SQL*Plus replacement. Since **SQLcl 26.1**, it ships with the entire APEXlang toolchain built in – no separate apexlang package exists.

⚠ **Gotcha:** SQLcl 26.1 bundles Java 16. The APEXlang compiler needs Java 17+. The first apex generate will crash with `UnsupportedClassVersionError`.

Give SQLcl a modern JVM.

```
$ bash scripts/04-install-java21.sh  
# installs openjdk@21 + appends JAVA_HOME/PATH to ~/.zshrc if missing
```

Two lines in ~/.zshrc. Tomorrow–you doesn't repeat the mistake. You'll see this in the recap – I count it as one step because the fix is the install.

It's already in your terminal.

```
$ bash scripts/05-start-sqlcl.sh
```

```
SQL> apex generate -name "Hello Webinar" -dir ./projects
```

```
Files created at ./projects/starter-app
```

```
SQL> apex validate -input ./projects/starter-app
```

```
Validation successful.
```

APEXlang is Oracle's new declarative APEX-as-code language, GA in APEX 26.1 (May 2026).

The compiler lives inside SQLcl. You get generate, validate, import, export -exptype

APEXLANG – the whole loop.

One browser round-trip, then done.

```
$ bash scripts/06-auth-oci.sh  
# refreshes existing session, or starts browser auth if none exists
```

Session token, not API key. Lives ~1 hour, refreshable. No public-key upload dance.
This is the only browser window we open until the app is live.

Give the agent Oracle's playbook.

```
$ bash scripts/06b-install-oracle-skills.sh
# github.com/oracle/skills → ~/.claude/skills/{apexlang, oracle-db, oracle-oci} – skips what's already there
✓ installed apexlang    templates/region-components/*  assets/grammar/apexlang.ebnf  tools/query-valid-props.mjs
```

Oracle publishes skills for coding agents: the APEXlang grammar, a canonical .apx template for every region type, and a tool that asks the compiler which properties a component accepts. Without them the agent reverse-engineers the compiler jar and guesses – that cost a whole night of unique constraint violated. With them it reads chart.bar.md and gets it right the first time. Restart the agent session so the skills load – then prompt.

Claude Code plugin route: /plugin marketplace add oracle/skills → /plugin install apex@oracle-skills. Other agents: npx skills add oracle/skills/apex.

One prompt. Plain English.

```
"Provision an Always Free Autonomous Database in my personal compartment.  
Wait for it to be AVAILABLE. Download the wallet. Create an APEX workspace  
called WEBINAR. Scaffold an APEXlang project called Sales Dashboard with a  
table sales(id, region, amount, sale_date), seed 20 sample rows, and add  
an interactive report page on it. Validate, import into the ADB, print the  
APEX URL, open it in the browser."
```

That's the whole demo, from the human. The next slide is what the agent runs – you watch, you interrupt if something looks wrong. Otherwise: silence and green checkmarks.

Under the hood.

```
# provision Always Free 19c ADB, wait AVAILABLE, download wallet
```

```
$ bash scripts/07-provision-adb.sh
```

```
# create WEBINAR schema + workspace + workspace-admin user
```

```
$ bash scripts/08-setup-workspace.sh
```

```
# validate + apex import + print URL + open browser
```

```
$ bash scripts/09-deploy-app.sh
```

```
# APEX Builder login (workspace / user / password) + app URL – any time, from the local dot-files
```

```
$ bash scripts/10-show-credentials.sh
```

Three idempotent scripts, plus one that prints the workspace login. Under a minute each once your ADB slot is warm. Each one prints ✓ for what it skips and → for what it actually does – safe to re-run mid-demo if something goes sideways.

The gotchas we hit so you don't have to.

- ▶ SQLcl 26.1 bundles Java 16, APEXlang needs 17+ – script 04 installs openjdk@21 and wires JAVA_HOME
- ▶ Session-token profiles need OCI_CLI_AUTH=security_token – every OCI-touching script sets it
- ▶ Free-tier quota lags 30–60s after termination – script 07 has a retry loop on QuotaExceeded
- ▶ SQLcl mangles multi-line PL/SQL from heredocs – script 08 writes SQL to a temp file with shell-substituted values
- ▶ installScript lives inside supportingObject as a component, not as an install {} block – see supporting-objects.apx
- ▶ Never author a region's attributes () – APEX auto-generates that singleton, declaring it double-inserts (WORKSHEETS_PK / JET_CHARTS_PK) – column, series, axis *and* the singleton's groups (chart { type: pie }, chartLayout { height: 400 }) go directly under region
- ▶ A chart with no chartLayout { height } renders 0px tall – blank page, clean logs – always set a height
- ▶ A chart region with no appearance { template } has no wrapper div – JET throws "Region element not found" and shows "No data to display" while the data call returns rows – give charts @/standard
- ▶ IR columns are created hidden unless declared; dataType is STRING|NUMBER|DATE, not VARCHAR2 – see p00001-home.apx
- ▶ Antigravity 2.x ships no CLI binary – script 01 detects the .app bundle directly

Every one of these cost me somewhere between 5 and 45 minutes the first time. They're all fixed in the scripts on this repo – clone and go.

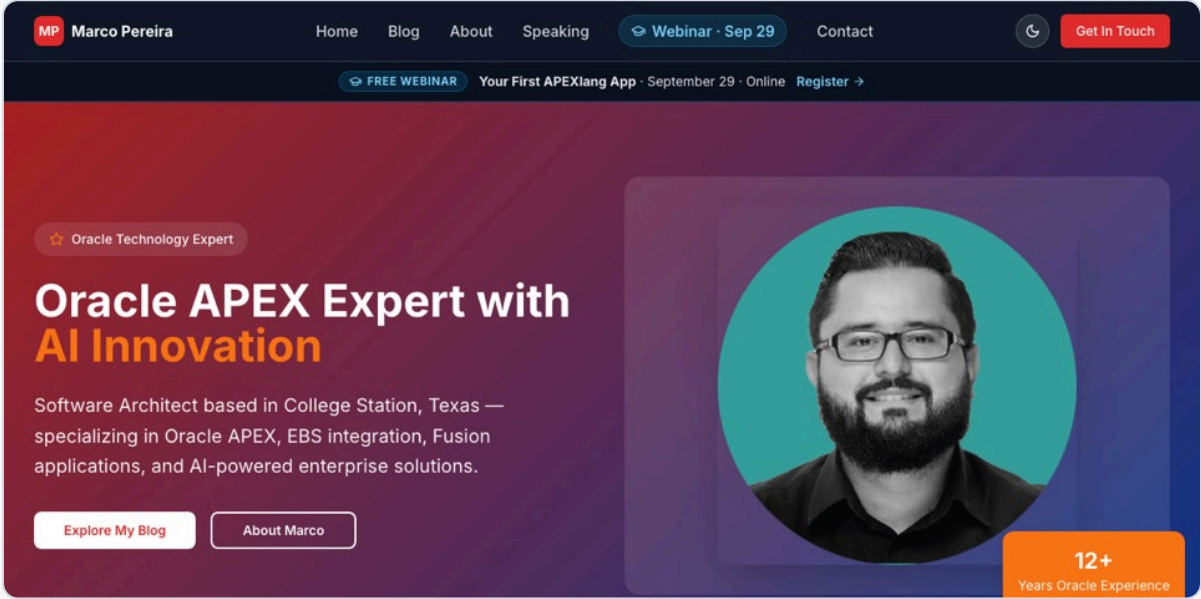
6 steps. 1 prompt. 1 live app.

1. `curl ... | sh` agent with a terminal (Antigravity)
 2. `brew install oci-cli` talk to Oracle Cloud
 3. `brew install --cask sqlcl` talk to Oracle DB + APEXlang
 4. `brew install openjdk@21` give SQLcl a modern JVM
 5. `oci session authenticate` sign in, session token
 6. `bash scripts/06b-install-oracle-skills.sh` Oracle's APEXlang grammar, templates, compiler queries
- `"...build me the app."` agent does the rest

That is the entire path from an empty Mac to a live Oracle APEX application, terminal-only, in under an hour.

Questions?

```
$ echo $CONTACT
Marco Pereira · markuspg.com
github.com/Markuspg1/your-first-apexlang-app
linkedin.com/in/marco-pereira-740022122
```



Grab the code
github.com/Markuspg1/your-first-a
pexlang-app



Follow me on LinkedIn
linkedin.com/in/marco-pereira-740
022122